

Mensageria com RabbitMQ

Objetivo

Implementar um sistema de mensageria utilizando RabbitMQ, compreendendo na prática o funcionamento de filas, durabilidade de mensagens e as semânticas de entrega *at-least-once* e *at-most-once*.

Contexto

Em sistemas distribuídos, o desacoplamento entre produtores e consumidores de mensagens é um padrão fundamental. O RabbitMQ é um message **broker** que implementa o protocolo AMQP e permite que mensagens sejam enfileiradas e entregues de forma confiável, mesmo quando consumidores estão temporariamente indisponíveis. Nesta atividade, você irá simular cenários reais de produção e consumo de mensagens, variando o comportamento do consumidor (*online/offline*) e a semântica de entrega.

Pré-requisitos

- RabbitMQ instalado localmente ou via Docker:
 - `docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management`
- Python 3 com a biblioteca `pika` instalada: `pip install pika`
- Interface de gerenciamento em `http://localhost:15672 (guest / guest)`

Questões

Questão 1 – Fila durável e consumidor offline

Implemente dois scripts Python: `produtor.py` e `consumidor.py`. O produtor deve publicar 5 mensagens em uma fila chamada `tarefas`. A fila deve ser configurada como **durável** e as mensagens devem ser marcadas como **persistentes**, garantindo que sobrevivam a uma reinicialização do broker.

Simule o seguinte fluxo:

- Execute o produtor com o consumidor ainda offline.
- Verifique na interface de gerenciamento que as mensagens estão acumuladas na fila (coluna *Ready*).
- Inicie o consumidor e observe o recebimento das mensagens acumuladas.

- Execute novamente o produtor com o consumidor já online e observe a entrega em tempo real.

Entregáveis: código-fonte dos dois scripts + capturas de tela mostrando (a) fila com mensagens aguardando (consumidor offline) e (b) consumidor recebendo as mensagens acumuladas após iniciar.

Questão 2 – Semântica at-least-once

Modifique o consumidor da Questão 1 para implementar a semântica **at-least-once**. Nesta abordagem, o ACK (confirmação) só é enviado ao broker **após o processamento bem-sucedido** da mensagem. Para simular falha, lance uma exceção nas mensagens de índice par e observe o broker reenviar a mensagem. Salve como `consumidor_alo.py`.

Pontos de atenção:

- Use `auto_ack=False` ao registrar o consumo.
- Em caso de erro, use `channel.basic_nack(requeue=True)` para recolocar a mensagem na fila.
- Em caso de sucesso, use `channel.basic_ack()` para confirmar o processamento.

Entregáveis: código-fonte + capturas de tela mostrando a mensagem sendo reenviada pelo broker após a falha simulada e, em seguida, processada com sucesso.

Questão 3 – Semântica at-most-once

Implemente a variante **at-most-once**. Nesta abordagem, o ACK é enviado automaticamente pelo broker no momento em que a mensagem é entregue ao consumidor, **antes do processamento**. Se o consumidor falhar ao processar, a mensagem é perdida permanentemente. Demonstre isso simulando uma falha após o recebimento. Salve como `consumidor_amo.py`.

Pontos de atenção:

- Use `auto_ack=True` ao registrar o consumo.
- Introduza uma exceção no callback para simular falha de processamento após a entrega.
- Verifique na interface do RabbitMQ que a mensagem foi removida da fila mesmo sem processamento bem-sucedido.

Entregáveis: código-fonte + capturas de tela mostrando a fila zerada após a entrega e o log de erro no terminal do consumidor.

Questão 4 – Análise comparativa

Com base na sua implementação, responda: em que tipos de sistema cada semântica é mais apropriada? Cite ao menos um exemplo de sistema real para cada uma e justifique sua escolha considerando as implicações de duplicação de mensagens versus perda de mensagens.

Orientações de Entrega

- Submeta um arquivo compactado (.zip) contendo todos os scripts Python e um relatório em PDF ou DOCX.
- O relatório deve conter as capturas de tela solicitadas em cada questão, com breve legenda explicando o que está sendo demonstrado.
- As capturas devem mostrar claramente o terminal do produtor/consumidor e, quando relevante, a interface de gerenciamento do **RabbitMQ** (`localhost:15672`).
- O código deve ser original. Implementações idênticas entre alunos implicam anulação da atividade para todos os envolvidos.